

DSA

Name: Muzammil Ahmed

Roll No: CT-24307

Sec: F

Lab 9:

Q1:

```
1  #include <iostream>
2  #include <stack>
3  using namespace std;
4  class TreeNode{
5  public:
6      int val;
7      TreeNode *left,*right;
8  TreeNode(int value){
9      val=value;
10     left=right=NULL;
11 }
12 };
13 class BinarySearchTree{
14 public:
15     TreeNode* root;
16     BinarySearchTree() : root(NULL) {}
17     void insert(int key){
18         root=insertRec(root,key);
19     }
20     void inorder(){
21         inorderRec(root);
22         cout<<endl;
23     }
24     void preorder(){
25         preorderIter(root);
26         cout << endl;
27     }
28     void postorder(){
29         postorderIter(root);
30         cout<<endl;
31     }
32     private:
33     TreeNode* insertRec(TreeNode* node, int key){
34         if (node ==NULL)
35             return new TreeNode(key);
36         if(key<node->val)
37             node->left = insertRec(node->left, key);
38         else if(key > node->val)
39             node->right = insertRec(node->right, key);
40         return node;
41     }
42     void inorderRec(TreeNode* root){
43         if (root!=NULL){
44             inorderRec(root->left);
45             cout<<root->val<<" ";
46             inorderRec(root->right);
47         }
48     }
```

```

49 void preorderIter(TreeNode* root){
50     if(root == NULL)
51         return;
52     stack<TreeNode*> s;
53     s.push(root);
54     while(!s.empty()){
55         TreeNode* node = s.top();
56         s.pop();
57         cout << node->val << " ";
58         if (node->right) s.push(node->right);
59         if (node->left) s.push(node->left);
60     }
61 }
62 void postorderIter(TreeNode* root){
63     if(root == NULL)
64         return;
65     stack<pair<TreeNode*, bool>> s;
66     s.push(make_pair(root, false));
67     while(!s.empty()){
68         pair<TreeNode*, bool> top = s.top();
69         s.pop();
70         if (top.second){
71             cout<<top.first->val <<" ";
72         }
73         else{
74             s.push(make_pair(top.first, true));
75             if (top.first->right) s.push(make_pair(top.first->right, false));
76             if (top.first->left) s.push(make_pair(top.first->left, false));
77         }
78     }
79 }
80 };
81 int main(){
82     BinarySearchTree bst;
83     bst.insert(50);
84     bst.insert(30);
85     bst.insert(20);
86     bst.insert(40);
87     bst.insert(70);
88     bst.insert(60);
89     bst.insert(80);
90     cout<<"Inorder: ";
91     bst.inorder();
92     cout<<"Preorder: ";
93     bst.preorder();
94     cout<<"Postorder: ";
95     bst.postorder();
96     return 0;
97 }

```

Output:

```

"F:\3rd semistor\dsa\labs\lab!  ×  +  v
Inorder: 20 30 40 50 60 70 80
Preorder: 50 30 20 40 70 60 80
Postorder: 20 40 30 60 80 70 50

Process returned 0 (0x0)    execution time : 0.236 s
Press any key to continue.

```

Q2

```

1  #include <iostream>
2  #include <string>
3  using namespace std;
4  class EmployeeNode{
5  public:
6      int id;
7      string name;
8      string department;
9      EmployeeNode* left;
10     EmployeeNode* right;
11     EmployeeNode(int i, string n, string d) : id(i), name(n), department(d), left(NULL), right(NULL) {}
12 }
13 class EmployeeBST{
14 public:
15     EmployeeNode* root;
16     EmployeeBST() : root(NULL) {}
17     void insert(int id, string name, string department){
18         if (isIDPresent(root, id)) {
19             cout << "Duplicate ID " << id << " skipped." << endl;
20             return;
21         }
22         root = insertRec(root, id, name, department);
23     }
24     bool search(int id){
25         EmployeeNode* found = searchRec(root, id);
26         if (found) {
27             cout << "Found: ID=" << found->id << ", Name=" << found->name << ", Dept=" << found
28             << "department" << endl;
29             return true;
30         } else {
31             cout << "Employee with ID " << id << " not found." << endl;
32             return false;
33         }
34     }
35     void deleteNode(int id){
36         if (!isIDPresent(root, id)) {
37             cout << "Employee with ID " << id << " not found. Deletion skipped." << endl;
38             return;
39         }
40         root = deleteRec(root, id);
41         cout << "Deleted employee with ID " << id << "." << endl;
42     }
43     void inOrderTraversal() {
44         cout << "Employees (sorted by ID):" << endl;
45         inOrderRec(root);
46         cout << endl;
47     }
48     void findMin() {
49         EmployeeNode* minNode = minValueNode(root);
50         if (minNode) {
51             cout << "Min ID Employee: ID=" << minNode->id << ", Name=" << minNode->name << ", Dept=" <<
52             minNode->department << endl;
53         } else {
54             cout << "Tree is empty." << endl;
55         }
56     }
57     void findMax() {
58         EmployeeNode* maxNode = maxValueNode(root);
59         if (maxNode) {
60             cout << "Max ID Employee: ID=" << maxNode->id << ", Name=" << maxNode->name << ", Dept=" <<
61             maxNode->department << endl;
62         } else {
63             cout << "Tree is empty." << endl;
64         }
65     }
66 private:
67     bool isIDPresent(EmployeeNode* node, int id) {
68         if (node == NULL) return false;
69         if (id == node->id) return true;
70         if (id < node->id) return isIDPresent(node->left, id);
71         return isIDPresent(node->right, id);
72     }
73     EmployeeNode* insertRec(EmployeeNode* node, int id, string name, string department) {
74         if (node == NULL) {
75             return new EmployeeNode(id, name, department);
76         }
77         if (id < node->id) {
78             node->left = insertRec(node->left, id, name, department);
79         } else if (id > node->id) {
80             node->right = insertRec(node->right, id, name, department);
81         }
82         return node;
83     }
84     EmployeeNode* searchRec(EmployeeNode* node, int id){
85         if (node == NULL || node->id == id) return node;
86         if (id < node->id) return searchRec(node->left, id);
87         return searchRec(node->right, id);
88     }
89     EmployeeNode* deleteRec(EmployeeNode* node, int id) {
90         if (node == NULL) return node;
91         if (id < node->id) {
92             node->left = deleteRec(node->left, id);
93         } else if (id > node->id) {
94             node->right = deleteRec(node->right, id);
95         } else {
96             if (node->left == NULL) {
97                 EmployeeNode* temp = node->right;
98                 delete node;
99                 return temp;
100             } else if (node->right == NULL) {

```

```

100         } else if (node->right == NULL) {
101             EmployeeNode* temp = node->left;
102             delete node;
103             return temp;
104         }
105         EmployeeNode* temp = minValueNode(node->right);
106         node->id = temp->id;
107         node->name = temp->name;
108         node->department = temp->department;
109         node->right = deleteRec(node->right, temp->id);
110     }
111     return node;
112 }
113 EmployeeNode* minValueNode(EmployeeNode* node) {
114     while (node && node->left != NULL) node = node->left;
115     return node;
116 }
117 EmployeeNode* maxValueNode(EmployeeNode* node) {
118     while (node && node->right != NULL) node = node->right;
119     return node;
120 }
121 void inOrderRec(EmployeeNode* node) {
122     if (node != NULL) {
123         inOrderRec(node->left);
124         cout << "ID=" << node->id << ", Name=" << node->name << ", Dept=" << node->department << endl;
125         inOrderRec(node->right);
126     }
127 }
128 };

```

```

129 int main() {
130     EmployeeBST empBST;
131     empBST.insert(101, "Muzammil", "HR");
132     empBST.insert(102, "Shazeeb", "IT");
133     empBST.insert(100, "Rehan", "Finance");
134     empBST.insert(103, "Saqib", "HR");
135     empBST.insert(102, "Fakhar", "Test");
136     empBST.inOrderTraversal();
137     empBST.search(102);
138     empBST.search(999);
139     empBST.findMin();
140     empBST.findMax();
141     cout << "\n--- Deleting Employee 102 ---\n";
142     empBST.deleteNode(102);
143     cout << "\n--- After Deletion ---\n";
144     empBST.inOrderTraversal();
145     return 0;
146 }

```

Output:

```
"F:\3rd semistor\dsa\labs\lab1" × + v
Duplicate ID 102 skipped.
Employees (sorted by ID):
ID=100, Name=Rehan , Dept=Finance
ID=101, Name=Muzammil, Dept=HR
ID=102, Name=Shazeeb, Dept=IT
ID=103, Name=Saqib, Dept=HR

Found: ID=102, Name=Shazeeb, Dept=IT
Employee with ID 999 not found.
Min ID Employee: ID=100, Name=Rehan , Dept=Finance
Max ID Employee: ID=103, Name=Saqib, Dept=HR

--- Deleting Employee 102 ---
Deleted employee with ID 102.

--- After Deletion ---
Employees (sorted by ID):
ID=100, Name=Rehan , Dept=Finance
ID=101, Name=Muzammil, Dept=HR
ID=103, Name=Saqib, Dept=HR

Process returned 0 (0x0)   execution time : 0.135 s
Press any key to continue.
```

Q3:

```
1  #include <iostream>
2  using namespace std;
3  class TreeNode{
4  public:
5      int val;
6      TreeNode *left,*right;
7  TreeNode(int value){
8      val=value;
9      left=right=NULL;
10 }
11 };
12 class BST{
13 public:
14     TreeNode* root;
15     BST(){
16         root=NULL;
17     }
18     TreeNode* insert(TreeNode* node, int key) {
19         if (node==NULL)
20             return new TreeNode(key);
21         if(key<node->val)
22             node->left = insert(node->left, key);
23         else if(key > node->val)
24             node->right = insert(node->right, key);
25         return node;
26     }
27     void insert(int key) {
28         root =insert(root, key);
29     }
30     TreeNode* findLCA(TreeNode* node, int t1, int t2) {
31         if(node == NULL)
32             return NULL;
33         if (t1 < node->val && t2 < node->val)
34             return findLCA(node->left, t1, t2);
35         if(t1 > node->val && t2>node->val)
36             return findLCA(node->right,t1,t2);
37         return node;
38     }
39 };
40 int main(){
41     BST bst;
42     bst.insert(90);
43     bst.insert(70);
44     bst.insert(40);
45     bst.insert(30);
46     bst.insert(80);
47     bst.insert(50);
48     bst.insert(40);
49     int t1=40, t2=30;
50     TreeNode* lca = bst.findLCA(bst.root, t1, t2);
51     if(lca != NULL)
52         cout<<"LCA of " << t1<<" and " << t2 <<" is:"<< lca->val<< endl;
53     else
54         cout<<"Nodes not found in the tree."<<endl;
55     return 0;
56 }
57
```

Output:

```
"F:\3rd semistor\dsa\labs\lab! X + v
LCA of 40 and 30 is:40

Process returned 0 (0x0) execution time : 0.129 s
Press any key to continue.
|
```

Q4:

```
1  #include <iostream>
2  using namespace std;
3  class TreeNode{
4  public:
5  int val;
6  TreeNode *left,*right;
7  TreeNode(int value){
8  val=value;
9  left=right=NULL;
10 }
11 };
12 class BST{
13 public:
14 TreeNode* root;
15 BST(){
16 root=NULL;
17 }
18 TreeNode* insert(TreeNode* node, int key){
19 if (node==NULL)
20 return new TreeNode(key);
21 if(key<node->val)
22 node->left = insert(node->left, key);
23 else if(key > node->val)
24 node->right = insert(node->right, key);
25 return node;
26 }
27 void insert(int key){
28 root =insert(root, key);
29 }
30 int sumNodes(TreeNode* node){
31 if(node == NULL)
32 return 0;
33 return node->val + sumNodes(node->left) + sumNodes(node->right);
34 }
35 };
36 int main(){
37 BST bst;
38 bst.insert(90);
39 bst.insert(70);
40 bst.insert(40);
41 bst.insert(30);
42 bst.insert(80);
43 bst.insert(50);
44 bst.insert(40);
45 int total =bst.sumNodes(bst.root);
46 cout<<"Sum of all nodes in the BST: "<<total<<endl;
47 return 0;
48 }
49
50
```

Output:

```
"F:\3rd semistor\dsa\labs\lab' X + v
Sum of all nodes in the BST: 360

Process returned 0 (0x0)   execution time : 0.145 s
Press any key to continue.
```

Q5:

```
create_key_node_in_bst.cpp  search_key_value_in_bst.cpp
1  #include <iostream>
2  #include <climits>
3  using namespace std;
4  class TreeNode {
5  public:
6      int val;
7      TreeNode *left,*right;
8      TreeNode(int value){
9          val=value;
10         left=right=NULL;
11     }
12 };
13 class BST {
14 public:
15     TreeNode* root;
16     BST(){
17         root=NULL;
18     }
19     TreeNode* insert(TreeNode* node, int key) {
20         if(node ==NULL)
21             return new TreeNode(key);
22         if (key < node->val)
23             node->left = insert(node->left, key);
24         else if (key > node->val)
25             node->right = insert(node->right, key);
26         return node;
27     }
28     void insert(int key){
29         root =insert(root, key);
30     }
31     void inorder(TreeNode* node, int &prev, int &minDiff){
32         if(node ==NULL)
33             return;
34         inorder(node->left, prev, minDiff);
35         if(prev!=-1)
36             minDiff=min(minDiff,node->val-prev);
37         prev=node->val;
38         inorder(node->right, prev, minDiff);
39     }
40     int getMinDiff(){
41         int prev =-1;
42         int minDiff =INT_MAX;
43         inorder(root, prev, minDiff);
44         return minDiff;
45     }
46 };
47
48 int main() {
49     BST bst;
50     bst.insert(50);
51     bst.insert(30);
52     bst.insert(20);
53     bst.insert(40);
54     bst.insert(70);
55     bst.insert(60);
56     bst.insert(80);
57     int result =bst.getMinDiff();
58     cout<<"Minimum difference between any two nodes: " << result << endl;
59     return 0;
60 }
```


Output:

```
"F:\3rd semistor\dsa\labs\lab' × + ∨  
Minimum difference between any two nodes: 10  
Process returned 0 (0x0)   execution time : 0.129 s  
Press any key to continue.
```